

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using **Linear Search**.

INPUT FORMAT

- First line contains integer N
- Second line contains N transaction IDs
- Third line contains suspicious transaction ID

OUTPUT FORMAT

Print: Fraud Transaction Found

EXAMPLE

Input

5

1201 1305 1450 1408 1502 1450

Output

Fraud Transaction Found

Linear Search!

Linear search is a searching technique in which each element of the array is checked one by one until the target element is found or array ends.

→ Algo!

1. Read the array elements.
2. Read the target element.
3. Traverse the array from left to right.
4. If the target is found, print "Fraud Transaction Found!"
5. Else, print "Transaction Not found".

→ Time Complexity!

- Best Case: $O(1)$
- Worst Case: $O(n)$

→ Space Complexity!

- $O(1)$

```
import java.util.Scanner;

public class LinearSearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        // Number of transactions
        int n = sc.nextInt();

        // Array creation
        int arr[] = new int[n];

        // Taking transaction IDs as input
        for(int i = 0; i<n; i++){
            arr[i] = sc.nextInt();
        }

        // Suspicious Transaction ID
        int target = sc.nextInt();

        for(int i = 0; i<n; i++){
            if (arr[i] == target) {
                System.out.println("Fraud
Transaction Founded");
                return;
            }
        }

        System.out.println("No Fraud
Transaction founded");
    }
}
```



OUTPUT

A screenshot of an IDE (IntelliJ IDEA) showing the execution of a Java program. The code defines a `LinearSearch` class with a `main` method that reads five integers and checks if they are in a fraud transaction list. The output shows the numbers 1201, 1305, 1450, 1408, and 1502, with the message "Fraud Transaction Founded".

```
public class LinearSearch {  
    public static void main(String[] args) {  
        int n = sc.nextInt();  
        // ...  
    }  
}
```

Command Prompt

```
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac LinearSearch.java  
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>java LinearSearch  
5  
1201  
1305  
1450  
1408  
1502  
1450  
Fraud Transaction Founded  
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>
```

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

Write a program using Binary Search.

Input Format

- First line contains N
- Second line contains sorted log IDs
- Third line contains target log ID

Output Format Print:

Log Found or Log Not Found

Constraints

- $1 \leq N \leq 10^5$

Example

Input

7

1001 1005 1010 1015 1020 1030 1040

1015

Output

Log Found

Binary Search:

Binary Search is an efficient searching algorithm used on sorted arrays. It repeatedly divides the search into two halves until the target element is found.

Working:

Find the middle element.
If middle element equals target, return Found.
If target is greater, search right half.

If target is smaller, search left half.

5. Repeat until found or search space becomes empty.

Time Complexity,

Best Case: $O(1)$

Worst Case: $O(\log n)$

Space Complexity!

→ $O(1)$

```
import java.util.Scanner;

public class BinarySearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        // Taking input for number of values
```

```
int n = sc.nextInt();

// Initialising the array
int arr[] = new int[n];

// Taking input for the array
for(int i = 0; i<n; i++){
    arr[i] = sc.nextInt();
}

// Taking input for target
int target = sc.nextInt();

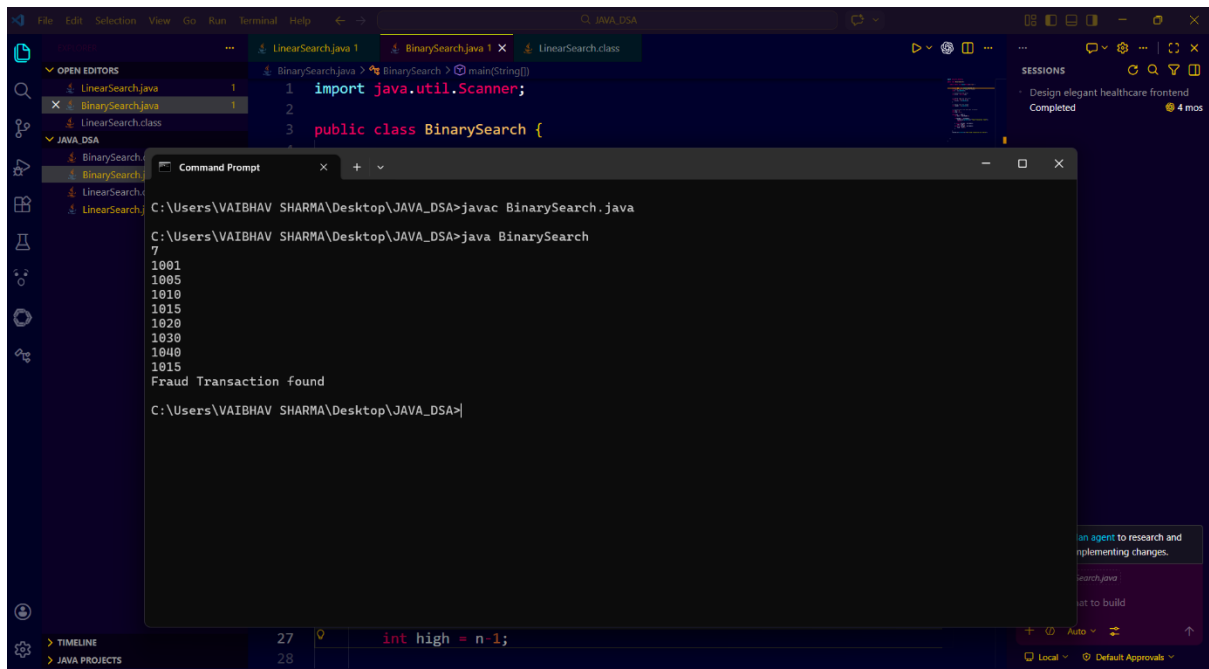
// Initialising (low and high) variables
int low = 0;
int high = n-1;

while(low <= high) {
    int mid = (low+high)/2;

    if(target == arr[mid]){
        System.out.println("Fraud
Transaction found");
        return;
    } else if(target > arr[mid]){
        low = mid+1;
    }else if (target < arr[mid]){
        high = mid-1;
    }
}
```

```
}  
    System.out.println("No fraud transaction  
are there");  
  
}  
  
}
```

OUTPUT



```
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac BinarySearch.java  
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>java BinarySearch  
7  
1001  
1005  
1010  
1015  
1020  
1030  
1040  
1015  
Fraud Transaction Found  
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>
```

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

Write a program using Bubble Sort.

Input Format

- First line contains N
- Second line contains salaries Output Format Print sorted salaries.

Example

Input

5

45000 32000 55000 28000 60000

Output

28000 32000 45000 55000 60000

Bubble Sort

→ Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position.

→ Algorithm

- Traverse the array
- Compare adjacent elements
- Swap if left element is greater than right element
- Repeat for all passes until array is sorted.

→ Time Complexity

Best Case : $O(n)$

Worst Case : $O(n^2)$

→ Space Complexity

$O(1)$

```
import java.util.Scanner;

public class BubbleSort {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Taking input for the number of unsorted
array;
        int n = sc.nextInt();

        // Initialising the array;
        int arr[] = new int[n];

        // Taking values of array as an input;
        for(int i = 0; i<n; i++){
            arr[i] = sc.nextInt();
        }

        // Sorting
        for(int i = 0; i<n-1; i++){
            for(int j = 0; j<n-i-1; j++){

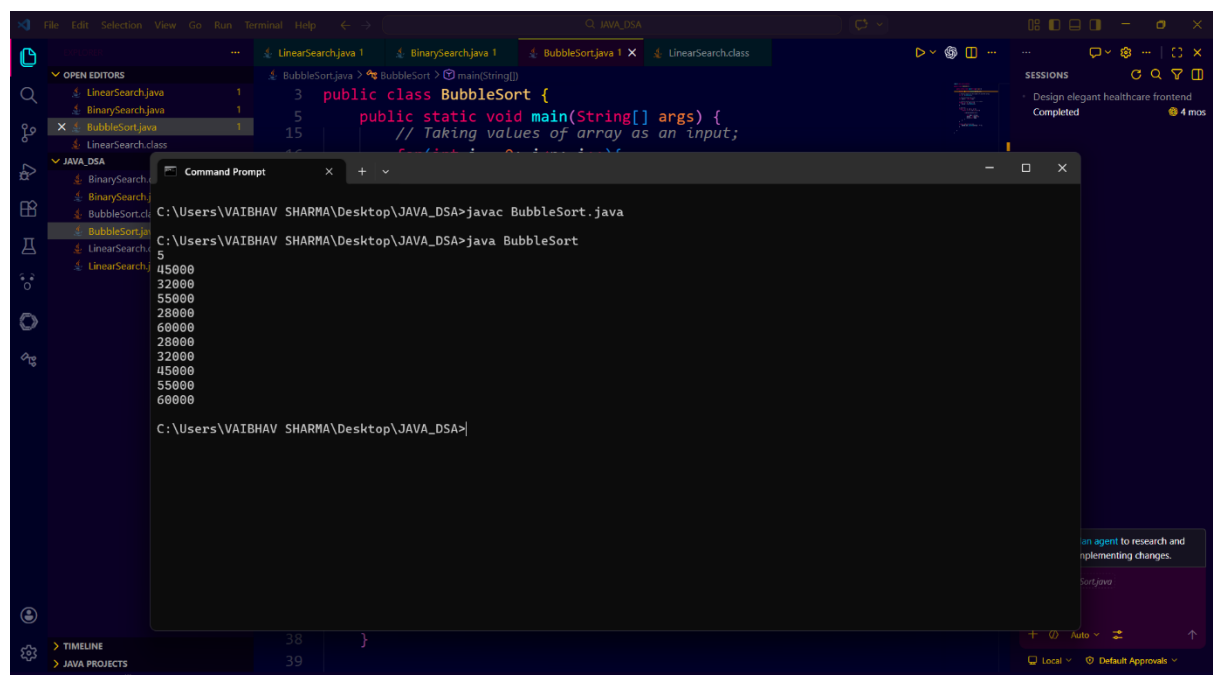
                if(arr[j]>arr[j+1]) {

                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;

                }
            }
        }
    }
}
```

```
        for(int i=0; i<n; i++){
            System.out.println(arr[i] + " ");
        }
    }
}
```

OUTPUT



```
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac BubbleSort.java
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>java BubbleSort
5
45000
32000
55000
28000
60000
28000
32000
45000
55000
60000
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>
```

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

Write a program using Insertion Sort.

Input Format

- First line contains N
- Second line contains response times Output Format Print sorted response times.

Example

Input

5

250 120 320 180 100

Output

100 120 180 250 320

Insertion Sort

- Insertion Sort builds a sorted portion of the array one element at a time by inserting each element into its correct position.

-> Algorithm

- > Assume first element is sorted
- > Pick next element as key
- > Compare key with previous elements
- > Shift larger elements to the right
- > Insert key at correct position
- > Repeat for all elements

-> Time Complexity

Best Case : $O(n)$

Worst Case : $O(n^2)$

-> Space Complexity

$O(1)$

```
import java.util.Scanner;

public class InsertionSort {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Taking input from user for the number of
arrays;
        int n = sc.nextInt();

        int arr[] = new int[n];

        // Taking input from user for the array
        for(int i=0; i<n; i++){
            arr[i] = sc.nextInt();
        }

        // Insertion Sort;

        for(int i = 1; i<n; i++){
            int key = arr[i];
            int j = i-1;

            while (j>=0 && arr[j]>key) {

                arr[j+1] = arr[j];
                j--;

            }
            arr[j+1] = key;
        }
    }
}
```

```

    }

    for(int i = 0; i<n; i++){
        System.out.println(arr[i] + " ");
    }
}
}

```

OUTPUT

The screenshot shows an IDE with the following components:

- File Explorer:** Lists files including LinearSearch.java, BinarySearch.java, BubbleSort.java, and InsertionSort.java.
- Code Editor:** Displays the InsertionSort.java file with the following code:


```

1 import java.util.Scanner;
2
3 public class InsertionSort {

```
- Command Prompt:** Shows the execution of the program:


```

C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac InsertionSort.java
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>java InsertionSort
250
120
320
180
100
100
120
180
250
320
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>

```
- Timeline:** Shows the execution flow of the program, with the current line of code being:


```

while (j>=0 && arr[j]>key) {

```

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Write a program using Selection Sort.

Input Format

- First line contains N
- Second line contains priority scores Output Format Print sorted priority scores.

Example

Input

5 9 2 8 1 5

Output

1 2 5 8 9

Selection Sort!

→ Selection Sort repeatedly finds the minimum element from the unsorted portion of the array and places it at the beginning.

→ Algorithm!

- Assume current index as min.
- Search remaining array for smaller element.
- update min index if found.
- Swap min element with current index.
- Repeat until array is sorted.

→ Time Complexity;

Best Case : $O(n^2)$

Worst Case : $O(n^2)$

→ Space Complexity!

$O(1)$

```
import java.util.Scanner;

public class SelectionSort {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        for(int i = 0; i<n; i++){

            arr[i] = sc.nextInt();
        }

        // Selection Sort;

        for(int i = 0; i<n-1; i++){
            int MinIndex = i;

            for(int j = i+1; j<n; j++){

                if(arr[j] < arr[MinIndex]){
                    MinIndex = j;
                }

            }

            int temp = arr[i];
            arr[i] = arr[MinIndex];
            arr[MinIndex] = temp;
        }
    }
}
```

```

    }

    for(int i = 0; i<n; i++){
        System.out.println(arr[i] + " ");
    }

}

}

```

OUTPUT

```

(c) Microsoft Corporation. All rights reserved.

C:\Users\VAIBHAV SHARMA>cd desktop
C:\Users\VAIBHAV SHARMA\Desktop>cd JAVA_DSA
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac SelectionSort.java
SelectionSort.java:33: error: cannot find symbol
    arrcmd[MinIndex] = temp;
    ^
  symbol:   variable arrcmd
  location: class SelectionSort
1 error

C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>javac SelectionSort.java
C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>java SelectionSort
5
9
2
8
1
5
1
2
5
8
9

C:\Users\VAIBHAV SHARMA\Desktop\JAVA_DSA>

```